

config.yaml

```
stages:
- {type: canny,      name: edges }
- {type: hough_lines, draw_on: source}
- {type: select,     input: edges }
- {type: contours,  draw_on: image}

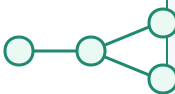
sinks:
- {type: ffmpeg, path: out.mp4}
- {type: image, path: e.png,
  input: edges }
```

`select` reaches back past a stage that consumed the frame

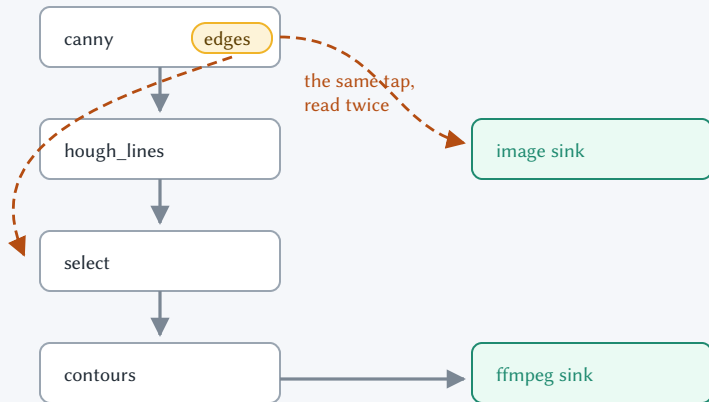
`hough\_lines` leaves a colour overlay in `ctx.image`, so the edge map is gone from the chain — but not from the taps. `select` rebinds `ctx.image` from one, using the same resolution rules a sink's `input:` uses.

Expressible: a tree

One producer, any number of consumers. Every stage has exactly one input, so the whole thing flattens to an ordered list plus `name:` taps — which is what the file already is, and why the editor shows a list.



what it means



Not expressible: a merge

Two producers feeding one stage. `Pipeline.run` carries a single `ctx.image` down an ordered list, so a stage has nowhere to receive a second input: masking this frame by that other branch, differencing two preprocessing variants, compositing an overlay onto a different base.



Making it possible means stages taking an `input:` too, named buffers instead of one `ctx.image`, and a topological sort — and a decision about what `ctx.store` means when two branches both write `contours`.

Until then: one chain, several outputs. Which is every config here so far.